

Syslab数据科学模块初探 强化学习工具箱

任品

苏州同元软控信息技术有限公司

2025年5月23日

课程须知

本课程课程目标:

本课程面向强化学习工具箱新用户，讲述强化学习的基本概念，并且在Syslab中演示如何构建完整的强化学习案例，帮助用户对强化学习建立基本了解，以及学会在 Syslab中使用强化学习工具箱进行强化学习模型的创建、训练、验证。

本课程基于云化版本：Syslab 2024b 构建

本课程需要在首选项加载:

TyBase

TyMath

TyPlot

TyReinforcementLearning

学习本课程之前需要学习:

Syslab基本功能

Julia语法

CONTENTS

目录

Part 1

强化学习简介

Part 2

强化学习基本概念

Part 3

强化学习主要组件

Part 4

Q-Learning介绍
及实践演示

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

Part1

强化学习简介

1. 定义与应用场景
2. 强化学习vs监督学习
3. 强化学习vs无监督学习

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

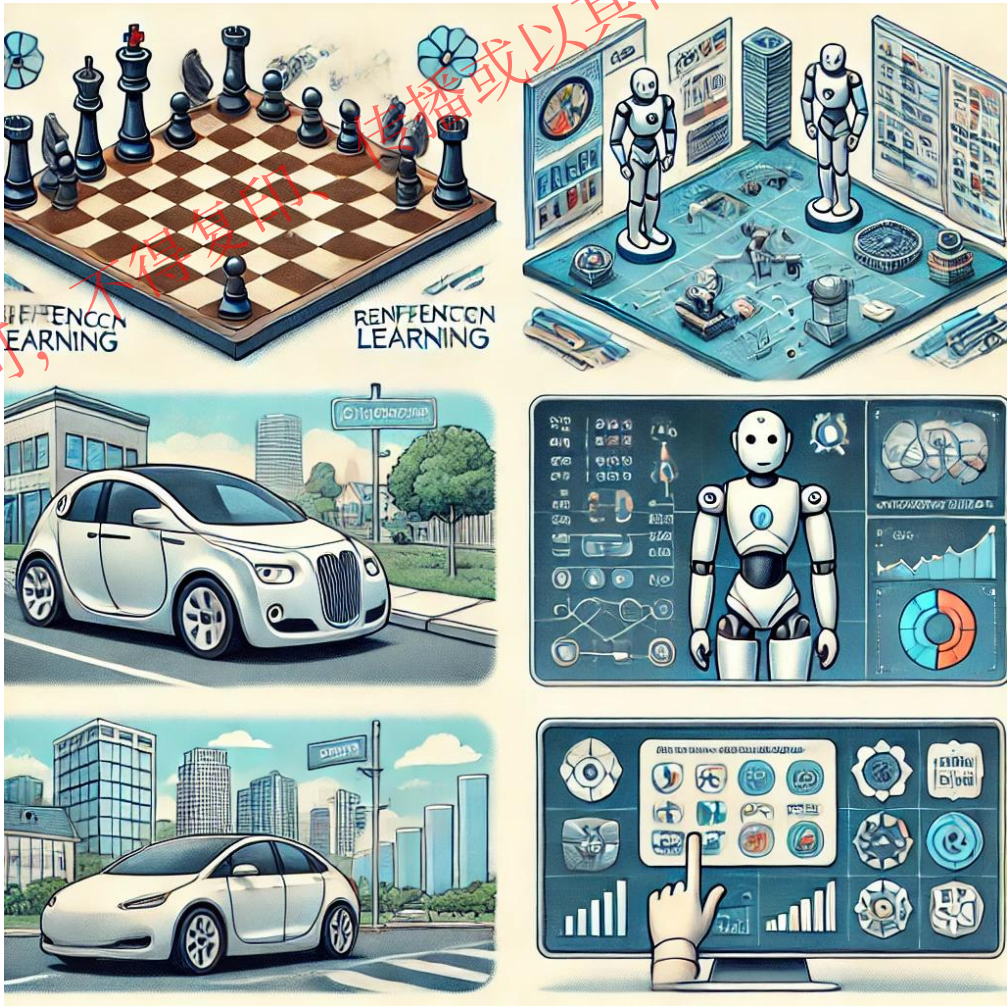
1.定义与应用场景

01 强化学习的定义

强化学习是一种**机器学习范式**，专注于如何在环境中做出一系列决策，以最大化某种**长期回报**。简单来说，就是一个智能体（Agent）在环境（Environment）中进行交互，根据自己的动作（Action）获得奖励（Reward），从而逐渐学会做出最优决策。

02 强化学习的应用领域

强化学习广泛应用于需要动态决策和优化的领域，如游戏、机器人控制、自动驾驶、金融交易等。这些应用展示了强化学习在处理不确定性和长期规划问题上的潜力。



2.强化学习vs监督学习

01 数据标注的差异

监督学习依赖于大量**精确标注**的数据来训练模型，每个输入样本都配有明确的输出标签。而强化学习则**不依赖预先标注**的数据，智能体通过与环境的互动自我学习，寻找最优策略。

02 目标的不同

监督学习的主要目标是让模型能够对新的、未见过的数据进行准确的**预测或分类**。相比之下，强化学习的目标是通过最大化累积奖励来**优化决策过程**，强调长期回报的最大化。

03 反馈机制的区别

在监督学习中，模型从标注数据中获得**即时且明确**的反馈，这有助于快速调整和优化模型性能。而在强化学习中，智能体接收到的反馈是环境中的奖励信号，这种反馈可能是**延迟的**，且**不一定直接指示最优行为**。



3.强化学习vs无监督学习



01 学习目标的差异

强化学习旨在通过与环境的互动，最大化长期累积奖励，强调**决策优化**；而无监督学习则专注于揭示数据的**内在结构或模式**，如聚类和降维，不依赖外部标签。

02 数据类型的不同

强化学习处理的是**状态-动作序列数据**，智能体在每个时间步长上根据当前状态选择动作并接收反馈，过程动态且依赖于当前状态所包含的关键信息；无监督学习则通常面对**静态数据集**，寻找数据内部的规律或结构。

03 应用场景的对比

强化学习广泛应用于需要**决策优化**的场景，如游戏、机器人控制等；无监督学习则适用于**数据探索和分析**领域，如市场细分、图像识别等，两者各有侧重。

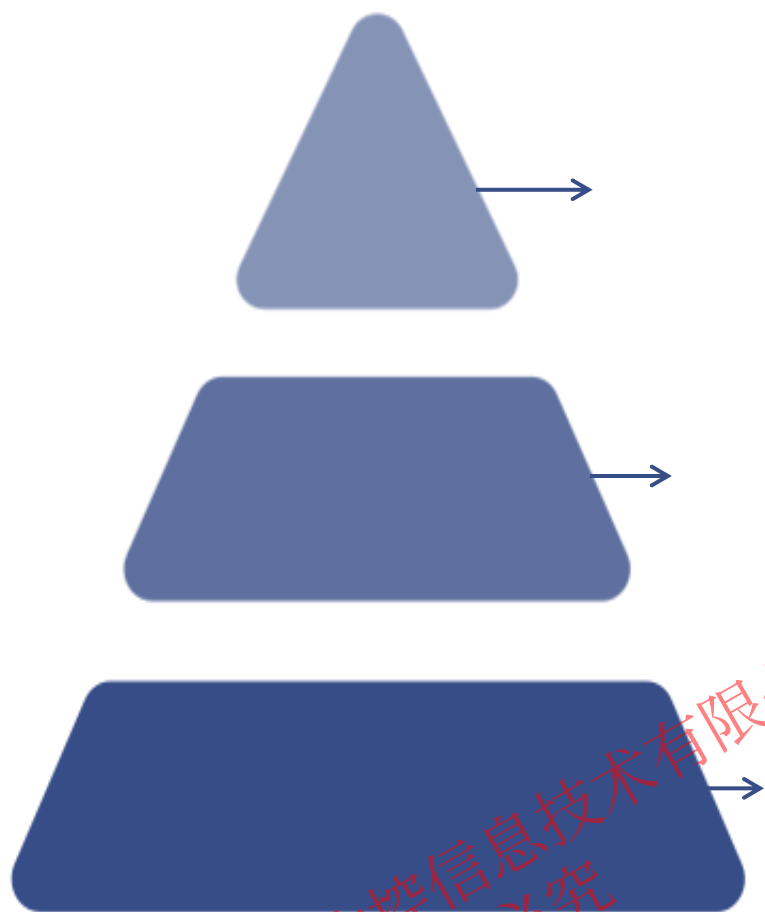
Part2

强化学习基本概念

1. 术语解释
2. 马尔科夫决策过程

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.术语解释



智能体的定义与作用

在强化学习中，**智能体**是**做出决策的实体**，可以是机器人、游戏中的角色或股票交易系统等。它通过**与环境的互动**来学习和优化其行为策略，以实现最大化累积奖励的目标。

状态、动作与奖励的关系

状态描述了智能体在某个时间点下对环境的**具体观察结果**，指导智能体做出决策；**动作**是智能体可以在环境中**执行的操作**，会改变环境的状态；**奖励**则是智能体在执行某个动作之后，从环境中获得的**反馈值**，用于评估动作的好坏。

环境的角色和功能

环境是智能体所处的**外部世界**，它根据智能体的行动做出反应，包括给智能体一个新的状态和相应的奖励。环境的设计对于智能体的学习过程至关重要，因为它决定了智能体能从中**获得的信息**。

2.马尔科夫决策过程（MDP）

Markov Decision Process（简称MDP）是一种数学模型，它描述了一个带有**随机性**的**序列决策过程**。MDP的假设是‘无记忆’或‘马尔科夫性质’，也就是说，未来状态的决定仅取决于当前状态和当前动作，而与之前的状态无关。

在MDP中，有四个关键要素：状态（S）、动作（A）、状态转移概率（P）和奖励函数（R）。



状态（S）

在MDP中，状态是智能体可能处于的所有**情况的集合**。每个状态代表了系统的一个特定配置或环境条件，为智能体的决策提供基础。



动作（A）

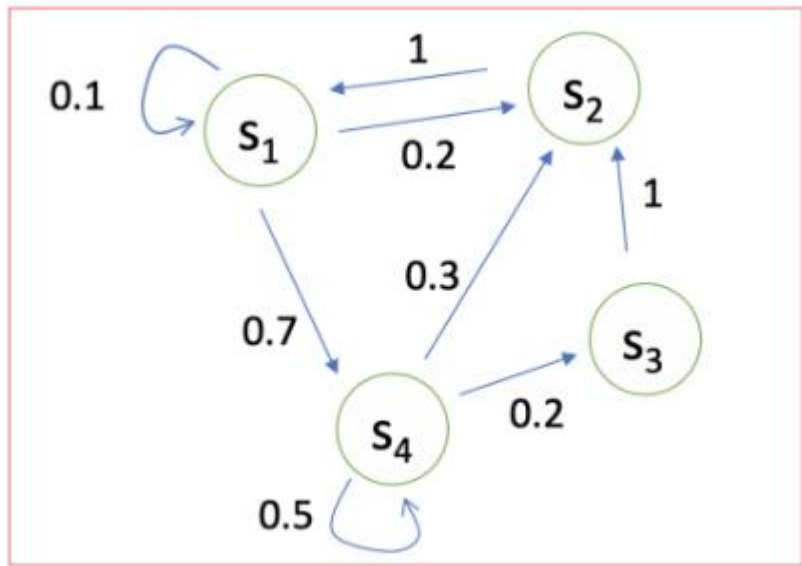
动作是指在给定状态下，智能体可以采取的所有**可能的操作**。这些操作旨在改变当前状态，推动系统向更优的状态转变，实现目标。



奖励函数（R）

奖励函数是MDP中用于评估智能体行为**好坏的标准**。它根据智能体在特定状态下采取的动作给予即时反馈，指导智能体学习最优策略。

2.马尔科夫决策过程 (MDP)



状态 (S)

即之前提到的状态集合，表示智能体可能处于的所有状态

状态转移概率 (P)

表示在执行某个动作后，从当前状态到下一个状态的概率。

奖励函数 (R)

是我们在某状态执行某个动作后所能获得的奖励，帮助智能体进行学习。

动作 (A)

表示智能体在不同状态下可以采取的所有可能的动作。

马尔科夫过程，加入奖励函数和动作之后，转化为马尔科夫决策过程

Part3

强化学习主要组件

1. 策略
2. 价值函数
3. 模型

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.策略

在强化学习中，**策略**是一个核心概念。它决定了智能体在每一个状态下该**采取什么样的动作**，简单来说就是智能体的‘行为规则’。策略可以分为两种类型：确定性策略（Deterministic Policy）和随机性策略（Stochastic Policy）。

01 确定性策略

确定性策略在**特定状态**下总是选择**同一动作**，如自动驾驶中左转决策。这种策略简单直接，易于理解和实现，适用于环境稳定、规则明确的场景。

03 策略的选择与应用

选择合适的策略类型对于强化学习至关重要。确定性策略适合简单环境，而随机性策略则更适合复杂或未知的环境，两者各有优势，需根据实际应用场景灵活选择。

02 随机性策略

随机性策略允许智能体在给定状态下**以概率方式**选择动作，适应复杂多变的环境。它通过探索不同动作来寻找最优解，特别适用于存在多个最优策略的情况。

2. 价值函数

除了策略之外，**价值函数**也是强化学习中不可或缺的组件。价值函数可以**衡量**一个状态或状态-动作对的好坏，从而帮助智能体评估当前状态和动作选择的效果。



01 状态价值函数 (State Value Function)

状态价值函数用于评估**在特定状态下**，遵循当前策略所能获得的**预期累积奖励**。它帮助智能体理解不同状态的价值，从而做出更优的决策。



02 状态-动作价值函数 (Action-Value Function)

状态-动作价值函数，也称为Q值，衡量**在给定状态下采取特定动作后**，根据当前策略执行一系列动作所能获得的**累积奖励**。它是强化学习中选择最优动作的关键依据。

3.模型

强化学习的方法还可以划分为**基于模型的方法** (Model-Based Methods) 和**无模型的方法** (Model-Free Methods) 。两者的主要区别在于是否显式地学习和利用环境的模型。

强化学习中，模型是用来**预测环境变化**的工具。它预测在某个状态下执行某个动作后，可能会转移到哪个新的状态，并且会获得怎样的奖励。简单来说，模型能够模拟或近似地描述环境的动态。

01 状态转移模型 (Transition Model)

预测在给定状态下执行某动作后，会转移到哪个状态。我们用 $P(s' | s, a)$ 表示状态转移模型，表示在状态 s 下执行动作 a ，转移到状态 s' 的概率。

02 奖励模型 (Reward Model)

预测在状态下执行某动作后会获得的奖励。我们用 $R(s, a, s')$ 表示奖励模型，表示在状态 s 执行动作 a 并转移到状态 s' 后获得的奖励。

策略告诉智能体如何**选择动作**，**价值函数**帮助智能体**评估状态和动作**的价值，而**模型**帮助智能体**预测环境的动态**。

Part4

Q-Learning 介绍及实践演示

1. 常见算法类别
2. Q-Learning 介绍
3. 悬崖漫步示例演示

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1. 常见算法类别

在强化学习领域，不同类型的算法各自发挥着独特的作用。我们通常将强化学习算法分为三大类：基于值函数、基于策略，以及结合型算法。

01 基于值函数的算法

这类算法通过学习状态或状态-动作对的值函数来进行决策。常见的算法包括 Q-Learning 和 Deep Q-Network (DQN)。

03 结合型算法

结合型算法将值函数和策略优化结合起来，以达到更稳定和高效的训练。常见的结合型算法包括 Actor-Critic 算法。

02 基于策略的算法

这类算法直接优化策略，以最大化累积奖励。其中包括 Policy Gradient Methods、Proximal Policy Optimization (PPO) 和 Trust Region Policy Optimization (TRPO)。

2. Q-Learning 介绍

01 Q-Learning 的定义

Q-Learning 是强化学习中非常经典的算法之一。它是一种无模型的、无策略的值函数估计方法，通过反复试验和观察，更新 Q 值表格（Q-Table），逐步学习最优策略。

02 Q-Learning 的核心思想

学习一种状态-动作价值函数（Q函数）即 $Q(s,a)$ 。它表示在某一状态下选择某个动作的预期累积奖励。通过不断更新Q值，智能体可以学会在不同状态下选择最优的动作，以期获得最大的长期回报。

03 Q-Learning 的目标

找到一个能够最大化累积奖励的策略。智能体会基于它所学的Q值表来做出决策。Q-Learning使用的策略通常是 ϵ -贪婪策略（ ϵ -greedy policy），这是一种平衡探索（Exploration）和利用（Exploitation）的策略。



2. Q-Learning 介绍 – 更新规则

Q-Learning更新Q值的公式如下：

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a) - Q(s,a) \right]$$

01

Q函数

智能体在状态 s 下选择动作 a 时的当前 Q 值，表示当前对这个动作的价值估计。

02

学习率 α

决定了每次迭代中 Q 值更新的幅度。一个恰当的学习率能够确保算法既不会因更新过快而错过最优解，也不会因更新过慢而降低学习效率。

03

即时奖励 r

智能体在当前状态 s 选择动作 a 后得到的即时奖励。这个奖励反映了执行动作后的直接效果

04

折扣因子 γ

反映了智能体对未来奖励的重视程度。若折扣因子接近 1，则表示智能体非常看重长期回报；若 γ 较小，则表示智能体更关心即时回报。

2. Q-Learning 介绍 – 学习过程

Q-Learning按照以下流程进行：

01 初始化

智能体开始于一个状态，并初始化Q值表格。表格中的所有Q值初始时一般设为0，或随机小值。

02 选择动作

根据当前状态和策略（通常是 ϵ -贪婪策略）选择一个动作。 ϵ -贪婪策略表示智能体以 $1-\epsilon$ 的概率选择 Q 值最大的动作，以 ϵ 的概率随机选择动作进行探索。

03 执行动作并观察结果

智能体执行所选择的动作，并从环境中获得新状态及奖励。

04 更新 Q 值

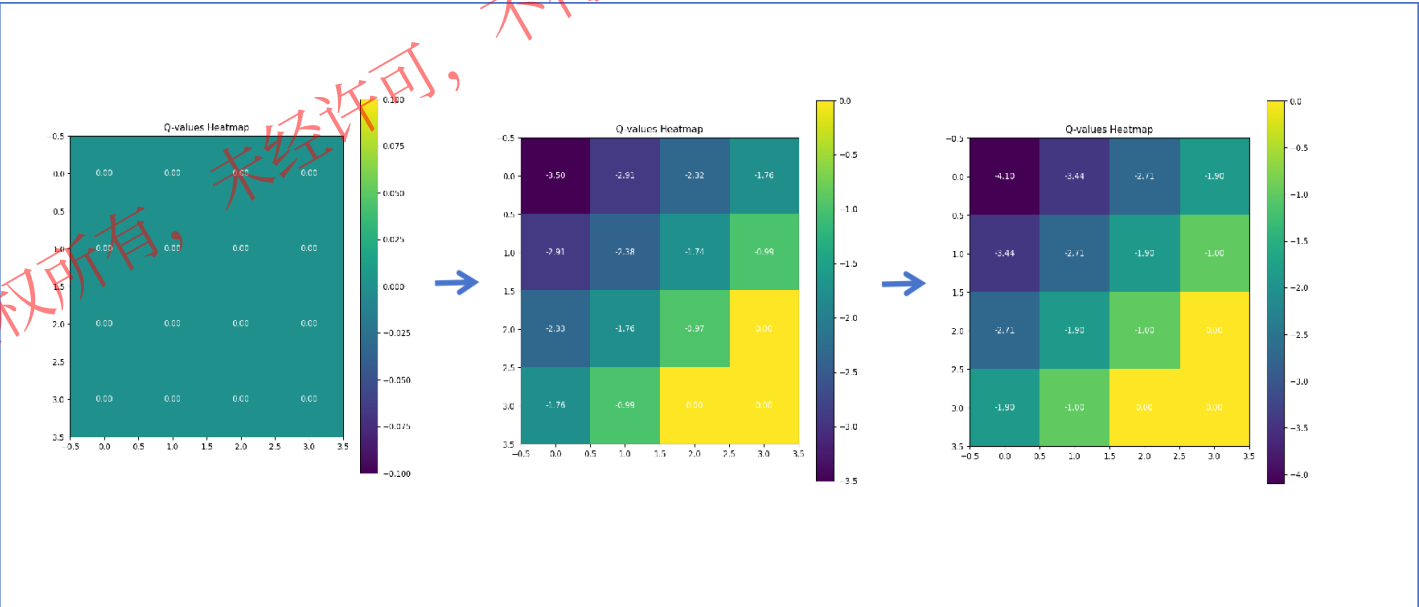
根据Q-Learning的更新规则调整Q值，使得其更好地反映实际的回报。

05 重复

返回步骤2，直到满足终止条件。智能体在反复交互中不断优化策略。

2. Q-Learning 介绍 – 学习过程

假设有一个网格，智能体位于网格的一个位置。它的目标是到达某一个指定的目标点。当智能体在网格中移动时，环境会给予即时奖励。到达目标点时，智能体得到正奖励；撞到墙壁或越界则会受到负奖励。



3.悬崖漫步示例演示

悬崖漫步 (CliffWalking) 又称为悬崖寻路问题，是强化学习的经典问题之一。悬崖漫步环境由一个 4 行 12 列的二维网格组成，智能体最初在一个网格的左下角中，终点位于右下角的位置，通过上下左右移动到达终点，每移动一步会得到-1单位的奖励，当智能体到达终点时游戏结束，但是空间中存在“悬崖”，若智能体进入“悬崖”则获得较大的负奖励并返回起点。

智能体在移动中有以下限制：

- (1) 智能体不能移出网格，如果智能体想执行某个动作移出网格，那么这一步智能体不会移动，但是这个操作依然会得到 -1 单位的奖励；
- (2) 如果智能体“掉入悬崖”，会立即回到起点位置，并得到 -100 单位的奖励；
- (3) 当智能体移动到终点时，该回合结束，该回合总奖励为各步奖励之和。



3.悬崖漫步示例演示

创建环境

```
using TyReinforcementLearning
env = BuildEnv("CliffWalking-v0")
```

创建智能体模型

```
models = rlQLModels(
    Qtable=zeros(StateDims(env),
        ActionDims(env)));
```

函数名	简介
rlQLModels	QL (Q learning) 智能体模型
rlSARSAModels	SARSA (State-Action-Reward-State-Action) 智能体模型
rlDQNModels	DQN (Deep Q-Network) 智能体模型
rlDoubleDQNModels	Double DQN (Double Deep Q-Network) 智能体模型
rlPGModels	PG (Policy Gradient) 智能体模型
rlACModels	AC (Actor-Critic) 智能体模型
rlDDPGModels	DDPG (Deep Deterministic Policy Gradient) 智能体模型



3.悬崖漫步示例演示

智能体选项用于设置智能体的各种参数，各个参数的意义参见帮助文档。

创建智能体选项

```
option = rlQLAgentOptions(  
    stateNum=StateDims(env),  
    actionNum=ActionDims(env))
```

函数名	简介
rlQLAgentOptions	QL (Q learning) 智能体选项
rlSARSAgentOptions	SARSA (State-Action-Reward-State-Action) 智能体选项
rlDQNAgentOptions	DQN (Deep Q-Network) 智能体选项
rldoubleDQNAgentOptions	Double DQN (Double Deep Q-Network) 智能体选项
rlPGAgentOptions	PG (Policy Gradient) 智能体选项
rlACAgentOptions	AC (Actor-Critic) 智能体选项
rlDDPGAgentOptions	DDPG (Deep Deterministic Policy Gradient) 智能体选项



3.悬崖漫步示例演示

使用模型与选项创建对应的强化学习智能体对象

创建智能体对象

```
agent = rlQLAgent(models,option);
```

函数名	简介
rlQLAgent	QL (Q learning) 智能体对象
rlSARSAgent	SARSA (State-Action-Reward-State-Action) 智能体对象
rlDQNAgent	DQN (Deep Q-Network) 智能体对象
rldoubleDQNAgent	Double DQN (Double Deep Q-Network) 智能体对象
rlPGAgent	PG (Policy Gradient) 智能体对象
rlACAgent	AC (Actor-Critic) 智能体对象
rlDDPGAgent	DDPG (Deep Deterministic Policy Gradient) 智能体对象



3.悬崖漫步示例演示

训练选项用于设置训练过程中的各种参数，各个参数的意义参见帮助文档。

创建智能体训练选项

```
train_options = rlTrainOptions(  
    max_episodes=1000,  
    learning_interval=1,  
    path=joinpath(pwd(),"result"));
```

语法

```
train_options = rlTrainOptions(  
    max_episodes=max_episodes,  
    batch_size=batch_size,  
    target_update_interval=target_update_interval,  
    soft_update=soft_update,  
    soft_update_factor=soft_update_factor,  
    learning_interval=learning_interval,  
    path=path,  
    save_condition=save_condition,  
    stop_condition=stop_condition,  
    averageing_window_length=averageing_window_length,  
    test=test,  
    evaluator=evaluator  
)
```



3.悬崖漫步示例演示

执行训练

```
result = train!(agent, env, train_options)
```

```
调试控制台  终端  端口 3  1: Julia REPL (v1.9.3)

正在初始化...
初始化完成!
julia> using TyReinforcementLearning

julia> env = BuildEnv("CliffWalking-v0")
PyObject <OrderEnforcing<PassiveEnvChecker<CliffWalkingEnv<CliffWalking-v0>>>>

julia> models = rlQLModels(
    Qtable=zeros(StateDims(env), ActionDims(env)));

julia> option = rlQLAgentOptions(
    stateNum=StateDims(env),
    actionNum=ActionDims(env),
    rlQLAgentOptions(epsilonGreedy(0.1, 0.001, 0.01), 0.9, 48, 4, 1000, TyReinforcementLearning.original_re
ds)

julia> agent = rlQLAgent(models,option);

julia> train_options = rlTrainOptions(
    max_episodes=1000,
    learning_interval=1,
    path=joinpath(pwd(),"result"));

julia> |
```

3.悬崖漫步示例演示

结果展示

```
env = BuildEnv("CliffWalking-v0", true)
opt = rlSimulationOptions(numSimulations=3)
rlSim(agent, env, opt)
CloseEnv(env)
```

调试控制台 终端 端口 3 1: Julia REPL (v1.9.3)

989 训练分数: -13 test分数: -13.0
990 训练分数: -13 test分数: -13.0
991 训练分数: -13 test分数: -13.0
992 训练分数: -13 test分数: -13.0
993 训练分数: -13 test分数: -13.0
994 训练分数: -15 test分数: -13.0
995 训练分数: -13 test分数: -13.0
996 训练分数: -13 test分数: -13.0
997 训练分数: -13 test分数: -13.0
998 训练分数: -13 test分数: -13.0
999 训练分数: -13 test分数: -13.0
1000 训练分数: -14 test分数: -13.0

Dict{Any, Any} with 9 entries:
"TrainOptions" => rlTrainOptions(1000, 256, 100, false, 0.05, 1, "/home/tongyuan/S
"average_test_scores_per_episode" => [-1000.0, -1000.0, -1000.0, -1000.0, -1000.0, -1000.0, -1000.0,
"BestScore" => -13.0
"steps_per_episode" => [723.0, 510.0, 144.0, 579.0, 203.0, 319.0, 189.0, 241.0, 143.0,
"test_scores_per_episode" => [-1000.0, -1000.0, -1000.0, -1000.0, -1000.0, -1000.0, -1000.0,
"average_steps_per_episode" => [723.0, 510.0, 144.0, 579.0, 203.0, 319.0, 189.0, 241.0, 143.0,
"rewards_per_episode" => [-2208.0, -906.0, -144.0, -975.0, -302.0, -319.0, -189.0, -241.0,
"total_agent_steps" => [723.0, 1233.0, 1377.0, 1956.0, 2159.0, 2478.0, 2667.0, 2908.0,
"average_rewards_per_episode" => [-2208.0, -906.0, -144.0, -975.0, -302.0, -319.0, -189.0, -241.0

julia> |



感谢您的聆听!

Thanks



未经授权，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究